

Epic 1 Summary And Final Test Plan

Date: 2026-07-03

Status: Epic 1 closeout package updated; E1.7 production-current guarded cloud review flow verified

Scope: E1.1-E1.7 knowledge base, PRD workflow, dashboard/Jarvis, final testmeeting rerun, critique

Executive Summary

Epic 1 built the internal ArchFlow proof engine: approved product material can become source inventory, context digest, PRD, task/backlog package, review report, KB update, and dashboard/Jarvis review packet without storing raw private material in public files.

The strongest result is method reliability, not market demand. Epic 1 proves that ArchFlow can operate a controlled PRD/KB workflow on itself and now has a production-current public dashboard plus hosted Jarvis APIs for guarded OpenRouter review packets. It does not prove that buyers will pay, that provider-backed Jarvis is safe to run by default, or that autonomous writeback is ready.

Epic 1 Scorecard

Dimension	Score	Visual	Evidence
Source boundary discipline	5/5	#####	E1.4 principle, public safety scan, testmeeting sanitizer
PRD artifact generation	5/5	#####	E1.2 and E1.2.8 PRD/PDF packages
KB governance	4/5	####.	WikiLLM run notes, E1.3 readback, E1.4 accepted principle
Dashboard/Jarvis review surface	5/5	#####	current Vercel public dashboard, same-origin API, Railway hosted API, and local dashboard packets
Cloud runtime independence	4/5	####.	Railway jarvis-api deployed and /health, CORS, PRD/ICP, agent-orchestra, role config, chat, and voice-safe text routes passed
Provider/model readiness	3/5	###..	guarded OpenRouter route works and blocks at missing approved server-side key; provider-backed calls remain gated
Market/customer proof	1/5	#....	No outreach, payment, or firm paid intent yet

Epic 1 Task State

Task	Status after this run	Evidence	Remaining gap
E1.1 Separate KB project	Done/reviewed	Project plan and public repo structure	None for current scope
E1.2 Dialogue to PRD cycle	Review, strong proof	June 26 proof and July 3 testmeeting rerun	Owner Done-state decision
E1.2.8 testmeeting PRD/PDF	Rerun locally on 2026-07-03	project/runs/E1.2/2026-07-02-testmeeting-local/	OpenRouter not rerun in this pass
E1.3 KB write/readback	Review	E1.3 readback run notes	Owner acceptance and future memory hygiene
E1.4 KB update principle	Accepted	project/reports/2026-07-03-kb-update-principle.md	Apply consistently

Task	Status after this run	Evidence	Remaining gap
E1.5 Case-study/content process	In progress/review	Reporting gate, content templates, dashboard reports	Final content pack after E2 evidence
E1.6 Role-split KB	Review	project/knowledge/ public-safe primary-operator and collaborator folders	Private vault wiring remains outside the public repo
E1.7 Hosted dashboard/Jarvis/API	Done for production-current guarded review flow; Review for full product runtime	Vercel production freshness, Vercel API, Railway hosted API, and dashboard API routing controls	Provider credential storage approval, auth hardening, persistent writeback, and raw voice storage remain later gates

Final Test Run: testmeeting

The final local test reran `project/scripts/e1_2_8_testmeeting_run.py` against the approved private fixture and methodology document. The public outputs remain sanitized derived artifacts only.

Test item	Result
Fixture present	Yes
Local runner completed	Yes
Stream events	6 events
OpenRouter status	not_run in this final baseline
Raw private text stored in public output	No by design; public output is sanitized
Main product focus	Release kickoff and discovery-to-delivery handoff modernization
Buyer hypothesis	Director/VP Product in B2B SaaS product team

E1.7 Final Hosted Proof

The final E1.7 run deployed the Vercel public dashboard/API and the `services/jarvis-api` Railway service root. Runtime provider mode is set to `openrouter`, but provider calls remain zero and execution blocks at the missing approved server-side key.

Check	Result	Evidence meaning
Railway service status	Passed	Latest service deployment reached running success state
Service domain	Passed	HTTPS Railway service domain generated and reachable
GET /health	Passed	status=ok, MODEL_PROVIDER=openrouter, provider calls 0, external writeback 0
CORS preflight	Passed	Vercel dashboard origin allowed for POST traffic
/api/chat	Passed	Review packet returned without model call
/api/config/roles	Passed	Agent-orchestra role config returned
/api/lanes/prd-icp	Passed	Architecture 1 service packet returned
/api/lanes/agent-orchestra	Passed	Architecture 2 control packet returned
/api/voice/chat	Passed	Text-only voice-safe packet returned; no raw audio storage
Recent HTTP error logs	Passed	No recent hosted HTTP errors returned for the validation probes

This proves the smallest cloud runtime needed for dashboard-to-backend review packets. It does not prove authenticated client portals, provider-backed generation, durable database writes, or autonomous Notion/GitHub/WikiLLM updates.

What The Setup Provides

For ArchFlow:

- a repeatable internal proof workflow;
- source-bound PRD generation;
- public-safe run reporting;
- a review-packet dashboard/Jarvis surface;
- WikiLLM run memory and provenance discipline;
- a clear bridge from Epic 1 proof to Epic 2 market research.

For clients:

- a service concept for turning messy product-team material into reviewed execution artifacts;
- artifacts that can include PRD, backlog, acceptance criteria, questions, risks, decisions, and KB handoff;
- a visible review/safety method rather than ungrounded AI output.

How To Use The Epic 1 Result

- Use E1.2.8 as the demo fixture for the PRD Pack method.
- Use E1.4 as the rulebook for all memory and evidence promotion.
- Use the dashboard/Jarvis review-packet surface as the operator cockpit.
- Use E2.0A as the gate before real public account research.
- Do not claim customer demand until E6/E7 produce paid or firm-intent proof.

Current Configuration Overview

Component	Current configuration	Safe default
Codex	local operator/editor/reviewer	Human-approved side effects
LangGraph	path/state/review architecture	Controller, not browser runtime claim
CrewAI	role structure and proof lane	Not default runtime
LlamaIndex	bounded retrieval layer	lexical/hybrid fallback, not memory
WikiLLM	durable public-safe memory	accepted facts only
Dashboard	static/operator UI	browser-local packets until reviewed
Jarvis API	hosted Railway guarded review contract plus Vercel same-origin API and local API-base control	provider calls zero
Railway	E1.7 guarded backend deployed and verified	no provider secret stored by this run, no writeback, no raw voice storage
Providers	OpenRouter route wired, approval-gated	MODEL_PROVIDER=openrouter, calls blocked until explicit external secret-storage approval

Critique

Epic 1 is strong as an internal operating proof and service-delivery method. It is still weak as a product proof. The smallest hosted backend now works, but the system is not yet a customer-ready SaaS because auth, persistence, client workspaces, provider budget ledgers, and buyer evidence remain unproven. The project should resist adding more dashboard surface until

E2 proves which buyer pain matters and which workflow should become the paid product.

Finish Criteria For Epic 1

Epic 1 can be considered complete for internal proof when:

- E1.4 accepted principle is applied.
- E1.2/E1.2.8 testmeeting artifacts pass validation.
- Epic 1 summary PDF is attached to Notion/GitHub.
- E1.6 role-split KB setup exists in the public-safe repo layer.
- E1.7 Railway provider-disabled runtime passes hosted health, CORS, and endpoint checks.
- Any remaining provider/writeback/auth/productization gap is explicitly moved to later follow-up, not hidden.

Epic 1 is now complete for the production-current guarded hosted review-packet runtime. It cannot be considered complete for provider-backed full product runtime until authentication, persistence, budget ledgering, provider activation, writeback approval, and customer-data handling are separately implemented and verified.

Notion Top-Of-Page Summary

Insert this at the top of the Epic 1 Notion page after push:

Epic 1 status, 2026-07-03: internal PRD/KB proof is complete for method scope. E1.4 is accepted. E1.6 now has a public-safe role